An Empirical Performance Benchmarking Evaluation of Locally Deployed Retrieval-Augmented Generation Constraints

Prajjwal Singh
Independent Technical Framework Study (Work in Progress)
Lucknow, India
prajjwals.acad@gmail.com

***Abstract*—This framework details a localized technical workbench designed to monitor the operational limits of privacy-preserving Retrieval-Augmented Generation (RAG) system pipelines on consumer hardware. Orchestrated via Python, LangChain, ChromaDB, and an offline Llama 3 instance running over local computing frames, the benchmark monitors the distinct processing impacts of token-based text segmentation boundaries. Our metrics isolate background resource overhead from inference execution loops. This document outlines an ongoing engineering study tracking structural system constraints rather than a generalized validated optimization conclusion.**

I. Experimental Methodology & System Boundary

To ensure reproducibility, our framework architecture decouples text ingestion, data index retrieval, and local synthesis text blocks:

- **Token Ingestion Routing:** Document text segments are extracted via PyPDF. To maintain literal linguistic parameter tracking, raw strings are divided using a token-aware context text splitter (`TokenTextSplitter`) calculated over the `all-MiniLM-L6-v2` vector model space.
- **Hardware Sandbox Specs:** All computation intervals were recorded locally on a consumer-grade laptop sandbox running macOS, Python, and localized Llama 3 model weights managed via Ollama daemon terminals.

II. Empirical Evaluation & Latency Segregation

Rather than combining pipeline performance measurements into an ambiguous sum, our benchmarking metrics track database initialization latencies completely separately from downstream model response extraction. The testing runs evaluate three distinct token window boundaries ($C \in \{200, 400, 600\}$ tokens), executing multiple continuous inference query trials per configuration block to log runtime cache fluctuations, while database vector indexing is performed exactly once per parameter tier.

A. Metrics Analysis & Baseline Limitations

Our logged data indicate that initialization overhead decreased from 5.404 seconds at 200-token chunks to 4.941 seconds at 400 tokens, followed by a slight increase to 5.094 seconds at 600 tokens due to a reduced volume of calculated vector nodes. Downstream generation latencies remained bounded between 0.787 and 1.120 seconds across the test segments.

TABLE I
Isolated System Latency Performance Averages Across Evaluated Token Scales

Token Size (Chunks)	Retrieval Depth (Top-K Depth)	One-Time Database Indexing Latency (s)	Mean Inference Latency (s)
200	3	5.404	1.120
400	3	4.941	0.787
600	3	5.094	0.889

While prompt context restrictions successfully forced the local model weights to output consistent fallback verification responses during unanswerable testing queries, these isolated test runs serve to map structural execution latency patterns rather than establish an automated numerical answer-quality evaluation metric or universal hallucination immunity.

III. Current Scope & Ongoing Research Directions

As an independent study in progress, the framework is bounded by its narrow dataset evaluation pool and specific hardware environment parameters. Future iterations will focus on integrating automated similarity metrics to log answer-quality deviations across varying model architectures.